

PROGETTI FINALI

Ultimo aggiornamento: 28 Luglio 2025 (Indicazioni per Terminal GUI, Joypad Action Client)

INDICAZIONI

Ciascun progetto deve essere svolto da un gruppo di persone di numero pari a quello indicato.

Per ciascuna traccia, si può fare riferimento ai seguenti supervisori.

- Roberto Masocco
- Prof. Daniele Carnevale
- Alexandru Cretu
- Alessandro Tenaglia

Gli studenti sono liberi di sviluppare le soluzioni che ritengono migliori per ogni traccia, anche ignorando i suggerimenti dati o aggiungendo ulteriori funzionalità.

Possiamo fornirvi ulteriori esempi di codice, spiegazioni e documentazione qualora vi sia utile.

Il progetto si considera completato quando il componente software richiesto presenta tutte le funzionalità richieste e soddisfa tutti i requisiti posti.

L'uso di AI per code generation deve essere **consapevole e responsabile**. È consigliato l'uso di AI Claude/ChatGPT per raccogliere spiegazioni e suggerimenti relativi ad API specifiche da usare, **a corredo della documentazione ufficiale**.

Quando è richiesta un'installazione di Linux, una VM **non** è sufficiente.

Come risorse generali a disposizione, si considerino

- tutto il materiale didattico fornito a lezione;
- [la documentazione ufficiale di ROS 2 Jazzy](#), ricca di esempi e tutorial sia in C++ che in Python;

- l'esempio [topic pubsub.py](#) di coppia publisher/subscriber in Python presente nel materiale didattico.

Per i progetti Terminal GUI e Joypad Action Client, è stato messo a disposizione degli studenti nei relativi repository un action/service server in Python utilizzabile per dei test preliminari. Una volta buildato il pacchetto 'test_server' (comando 'cbuild' o, equivalentemente essendone il primo un alias, 'colcon build --symlink-install', e poi '.install/local_setup.zsh'), esso può essere avviato con 'ros2 run test_server test_server'. Il codice è interamente in Python e contenuto in un unico file, e tutte le callback d'interesse per servizi e azioni sono esplicitate: per svolgere i relativi test, è sufficiente creare dei client per servizi e azioni dello stesso tipo e con lo stesso nome e cambiare alcuni valori di ritorno per testare i vari comportamenti possibili lato server (e.g. goal accepted/rejected, success/abort) (alcune righe commentate sono già presenti).

REQUISITI GENERALI

- Ogni progetto richiede l'implementazione di un **pacchetto di software ROS 2, in C++ o in Python.**
- Ogni package ROS 2 dovrà contenere, organizzati opportunamente secondo la struttura mostrata a lezione (si veda esempio di codice sui parametri nella Lezione 5):
 - codice sorgente;
 - config file;
 - launch file;
 - gestione parametri tramite librerie
 - [params_manager_cpp](#) per i package in C++;
 - [params_manager_py](#) per i package in Python;
 - compilazione nodo come component (solo per package C++).
- Tutti i progetti dovranno essere svolti sotto version control, entro repository Git hostate su **GitHub** a cui vi verrà fornito l'accesso dopo un setup iniziale (comprendente, e.g., le interfacce ad-hoc ed eventuali dipendenze aggiuntive necessarie). **Comunicare a R. Masocco**
 - username ed email GitHub dei componenti di ogni gruppo;
 - il progetto scelto da ciascun gruppo.

- Tutti i progetti dovranno essere svolti in ambiente di lavoro gestito tramite **Docker**, usando il framework **Distributed Unified Architecture (DUA)** in uso nel laboratorio. Gli ambienti saranno forniti inizialmente pronti per essere usati. Per approfondirli, sono disponibili le seguenti risorse:
 - repository GitHub [dua-template](#) (su cui saranno basate quelle dei progetti);
 - quick-start guide [dua-template](#);
 - [tesi di Dottorato di R. Masocco](#) capp. 4, 5.1 (cap. 3 costituisce un “libro di testo” per gli argomenti trattati durante le lezioni).

- LivoxLidarDetectMode
 - Servizio di enable/disable (std_srvs/srv/SetBool)
 - Servizio di reboot (std_srvs/srv/Empty) usando le API dell'SDK.
- Parametri
 - autostart (BOOL)
 - udp.ip (STRING)
 - udp.port (INTEGER)
 - publish_control_msgs (BOOL)
 - livox_info.pcl_data_type (INTEGER)
 - livox_info.scan_pattern (INTEGER)
 - livox_info.work_mode (INTEGER)
 - livox_info.work_mode_after_reboot (INTEGER)

I test verranno eseguiti dagli studenti in laboratorio sul LiDAR Mid-360 in dotazione.

ROS2 RESOURCE MONITOR (2 persone) (C++)

Realizzazione di un nodo ROS 2 che monitora e pubblica informazioni sullo stato del sistema.

- Funzionalità richieste
 - Servizi
 - Servizio di enable/disable (std_srvs/srv/SetBool)
 - Routine per il campionamento delle informazioni sullo stato del sistema
 - Percentuale di utilizzo CPU
 - Si legga la prima riga del file '/proc/stat', avente formato
"cpu user nice system idle iowait irq softirq steal
guest guest_nice"
 - Si calcoli il tempo totale come
user + nice + system + idle + iowait + irq + softirq
+ steal
 - Si calcoli il tempo di attività come
user + nice + system + irq + softirq + steal
 - Disponendo dei tempi campionati all'istante
precedente (attenzione alla gestione dei calcoli al
primo istante!), si calcolino
$$\text{total_delta} = \text{current.total()} - \text{prev_cpu_times_total}();$$
$$\text{active_delta} = \text{current.active()} - \text{prev_cpu_times_active}();$$
 - L'utilizzo di CPU all'istante corrente è dato da
$$\text{cpu_percent} = (\text{total_delta} > 0) ? (100.0 * \text{active_delta} / \text{total_delta}) : 0.0$$
 - Percentuale di utilizzo RAM
 - Si leggano, dal file '/proc/meminfo' e memorizzando i
valori in kB riportati, le righe
 - 'MemTotal'
 - 'MemAvailable'
 - La memoria correntemente usata, in kB, è data da
used_memory = mem_total - mem_available
da cui segue la percentuale

- Timer (periodo dato da parametro, si veda a seguire): la callback, ad ogni scatto, esegue la routine di campionamento e pubblica i dati calcolati
- Publishers
 - ~/cpu_usage: percentuale calcolata come sopra (std_msgs/msg/Float64)
 - ~/memory_usage: percentuale calcolata come sopra (std_msgs/msg/Float64)
- Parametri
 - autostart (BOOL)
 - sampling.period (INTEGER) (milliseconds)

I test verranno eseguiti in autonomia dagli studenti sulle loro macchine, o su macchine del laboratorio in caso di necessità.

JOYPAD ACTION CLIENT (2 persone) (C++) (Ubuntu Linux 24.04 host necessario)

Implementazione di un nodo ROS 2 che funga da action client in risposta alla pressione di determinati tasti su un joystick USB (estendendo le funzionalità offerte dai 'joy::GameController' e 'teleop_twist_joy::TeleopTwistJoy', parte di un'installazione di base di ROS).

- Funzionalità richieste
 - Azioni
 - Client azioni con [simple actionclient cpp](#)
 - Elenco client da implementare
 - Arm ([dua hardware interfaces](#)/action/Arm)
 - Disarm ([dua hardware interfaces](#)/action/Disarm)
 - Takeoff ([dua aircraft interfaces](#)/action/Takeoff)
(aggiungere al goal l'altitudine da parametro)
 - Landing ([dua aircraft interfaces](#)/action/Landing)
(aggiungere al goal l'altitudine di decisione da parametro)
 - Subscriber
 - Topic /joy (sensor_msgs/msg/Joy)
 - La callback esamina il contenuto dell'array 'buttons' e, quando rileva una entry corrispondente ad un bottone premuto, controlla se è uno di quelli noti dai parametri e in caso affermativo invoca la relativa azione con l'opportuno client.
 - Il goal, prima che l'azione venga chiamata, deve essere opportunamente popolato quando necessario:
 - Takeoff
 - takeoff_pose.header.frame_id = "map"
 - takeoff_pose.header.stamp = get_clock()->now()
 - takeoff_pose.pose.position.z = parametro (si veda a seguire)
 - Landing
 - descend = false

- L'esito dell'azione contenuto nel result (messaggio [dua common interfaces](#)/msg/CommandResultStamped) va stampato a schermo con una stampa di log di gravità opportuna (RCLCPP_WARN se SUCCESS, RCLCPP_ERROR se FAILED, RCLCPP_FATAL se ERROR)
- Parametri
 - actions.arm.name (STRING)
 - actions.arm.button (INTEGER)
 - actions.disarm.name (STRING)
 - actions.disarm.button (INTEGER)
 - actions.takeoff.name (STRING)
 - actions.takeoff.button (INTEGER)
 - actions.takeoff.altitude (DOUBLE) (meters)
 - actions.landing.name (STRING)
 - actions.landing.button (INTEGER)

I test verranno eseguiti con un dummy action server che verrà messo a disposizione nel repository, e alla fine su un drone simulato/reale. Agli studenti verrà fornito un joystick compatibile (qualora non ne abbiano già uno) e le istruzioni su come interfacciarlo da ROS 2.